Facial Landmark Detection

Facial Landmark Detection
1. Content Description
2. Program Startup
3. Core Code Analysis

1. Content Description

This lesson implements acquiring the color image and using the MediaPipe framework to detect the face. This section requires entering commands in the terminal, where the terminal is opened according to the mainboard type. This lesson uses the Raspberry Pi 5 as an example.

For the Raspberry Pi 5 mainboard, you need to open a terminal on the host machine, then enter the command to access the Docker container, and enter the commands mentioned in this lesson inside the terminal after entering the Docker container. For the tutorial on entering the Docker container from the host, please refer to **【Robot Configuration and Operation Guide】, section 【3.Enter Docker (For Raspberry Pi)】** in this product tutorial.

For the Orin and RDK mainboards, simply open a terminal and enter the commands mentioned in this lesson.

2. Program Startup

The Raspberry Pi PI5 controller needs to enter the Docker container first; RDK X5 and Orin controllers do not need to.

To enter the Docker container (for steps, refer to 【Docker Course】 --- 【4. Docker Startup Script】),

All the docker terminals for the commands below need to enter the same Docker container before executing **(for steps, refer to 【Docker Course】 --- 【3.Docker Submission and Multi-Terminal Access】)**

First, enter the following command in the terminal to start the robot camera and low-level driver environment:

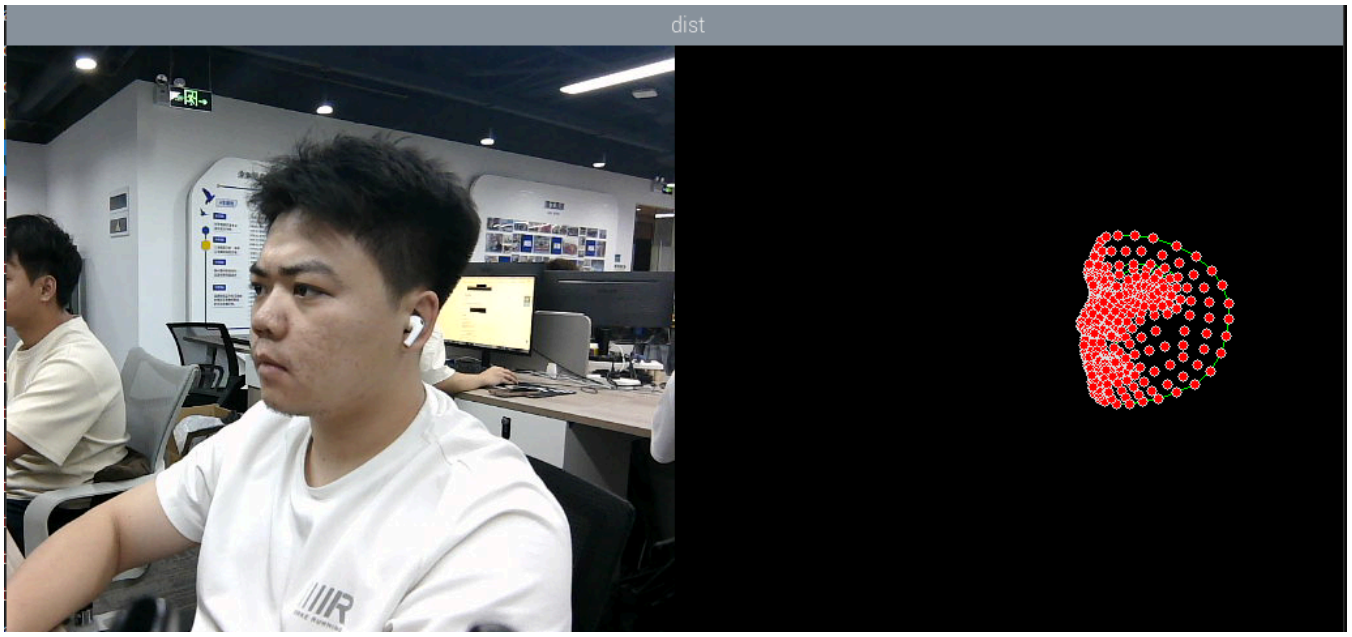
```
ros2 launch dofbot_pro_driver A1pro_camera_arm.launch.py
```

> **Note**
>
> This command is the official all-in-one startup script for the A1Pro, which automatically launches the image stream node according to the camera hardware configuration (USB monocular camera or Nuwa 3D depth camera) and loads the low-level drivers. If you only need to start the camera stream separately, you can also run `ros2 launch usb_cam camera.launch.py` (USB camera) or `ros2 launch ascamera hp60c.launch.py` (Nuwa camera).

After the camera is started successfully, open another terminal and enter the following command in the terminal to start the facial detection program:

```
ros2 run yahboomcar_mediapipe 04_FaceMesh
# Control the robotic arm to look forward
ros2 topic pub /arm6_joints dofbot_pro_interface/msg/ArmJoints "{joint1: 90, joint2: 180,
joint3: 0, joint4: 0, joint5: 90, joint6: 0, time: 1000}" --once
```

After the program runs, as shown in the figure below, the detected facial points will be displayed on the right side of the image:



3. Core Code Analysis

Program code path:

- Raspberry Pi 5 mainboard

  The program code is located in the running Docker. The path in the Docker is `/root/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/04_FaceMesh.py`

- Orin mainboard

  The program code path is `/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/04_FaceMesh.py`

- RDK mainboard

  The program code path is `/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/04_FaceMesh.py`

Import the required library files:

```
import time
import rclpy
from rclpy.node import Node
import cv2 as cv
import numpy as np
import mediapipe as mp
from geometry_msgs.msg import Point
from yahboomcar_msgs.msg import PointArray
from sensor_msgs.msg import Image
from cv_bridge import CvBridge, CvBridgeError
import os
```

Initialize the data and define the Publisher and Subscriber:

```
def __init__(self, staticMode=False, maxFaces=2, minDetectionCon=0.5, minTrackingCon=0.5):
    super().__init__('face_mesh_detector')
    self.mpDraw = mp.solutions.drawing_utils
    # Use the class in the mediapipe library to define a face object
    self.mpFaceMesh = mp.solutions.face_mesh
    self.faceMesh = self.mpFaceMesh.FaceMesh(
    static_image_mode=staticMode,
    max_num_faces=maxFaces,
    min_detection_confidence=minDetectionCon,
    min_tracking_confidence=minTrackingCon )
    # Define the attributes of the joint connection lines, which will be used in the
subsequent joint connection function
    self.lmDrawSpec = mp.solutions.drawing_utils.DrawingSpec(color=(0, 0, 255),
thickness=-1, circle_radius=3)
    self.drawSpec = self.mpDraw.DrawingSpec(color=(0, 255, 0), thickness=1, circle_radius=1)
    self.bridge = CvBridge()
    # Define the Subscriber for the color image topic
    camera_type = os.getenv('CAMERA_TYPE', 'usb')
    topic_name = '/ascamera_hp60c/camera_publisher/rgb0/image' if camera_type == 'nuwa' else
'/usb_cam/image_raw'
    self.subscription = self.create_subscription(
    Image,
    topic_name,
    self.image_callback,
    10)
```

Color image callback function:

```
def get_RGBImageCallBack(self,msg):
    # Convert the color image message data into image data using CvBridge
    frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')
    # Pass the obtained image into the defined pubFaceMeshPoint function; draw=False means
not drawing the joint points on the original color image
    frame, img = self.pubFaceMeshPoint(frame, draw=True)
    # Merge the two images
    dist = self.frame_combine(frame, img)
    key = cv2.waitKey(1)
    cv.imshow('dist', dist)
```

The pubFaceMeshPoint function:

```
def pubFaceMeshPoint(self, frame, draw=True):
    # Create a new image based on the size of the input image; the data type of the image is
uint8
    img = np.zeros(frame.shape, np.uint8)
    # Convert the color space of the input image from BGR to RGB for subsequent image
processing
    imgRGB = cv.cvtColor(frame, cv.COLOR_BGR2RGB)
    # Call the process function in the mediapipe library for image processing; during init,
the self.faceMesh object was created and initialized
    self.results = self.faceMesh.process(imgRGB)
    # Determine whether self.results.multi_face_landmarks exists, that is, whether a face
has been detected
    if self.results.multi_face_landmarks:
        for i in range(len(self.results.multi_face_landmarks)):
            if draw: self.mpDraw.draw_landmarks(frame, self.results.multi_face_landmarks[i],
self.mpFaceMesh.FACEMESH_CONTOURS, self.lmDrawSpec, self.drawSpec)
            # Connect each joint point on the blank image created earlier
            self.mpDraw.draw_landmarks(img, self.results.multi_face_landmarks[i],
self.mpFaceMesh.FACEMESH_CONTOURS, self.lmDrawSpec, self.drawSpec)
    return frame, img
```

The frame_combine function for merging images was mentioned in the first lesson of this chapter; please refer to the analysis of this function in 【Mediapipe Visual Course】 - 【1. Hand Detection】.